SeldonH — Mémo technique

Fonctionnement général de l'application

Philippe Dufresne

30 octobre 2026

Résumé

SeldonH est une application Qt/C++ de dialogue avec un modèle de langage (local par llama.cpp, ou distant par une API compatible OpenAI) destinée aux archéologues et géologues. Elle réalise des datations simples, calcule des Events au sens de ChronoModel, produit des fichiers de modèle `.chr` et aide à interpréter les résultats. Le principe directeur est de *séparer le langage du calcul* : le modèle de langage comprend la question, choisit et commente ; tous les calculs (calibrations, MCMC, fichiers) sont faits par du code C++ déterministe, porté de ChronoModel, sur l'ordinateur de l'utilisateur. Ce mémo décrit l'architecture (graphe fonctionnel), l'algorithme général de traitement d'une question et les principaux modules.

Table des matières

1 Principes de conception **1**

2 Graphe fonctionnel **2**

3 Algorithme général **2**

4 Sélection des compétences **5**

5 Moteurs de langage **5**
5.1 Modèle local (llama.cpp) 5
5.2 Modèle distant 5

6 Calculs de datation **6**
6.1 Dates isolées 6
6.2 Calcul d'un Event (MCMC) 6
6.3 Autres calculs 6

7 Création d'un modèle ChronoModel **6**

8 Interprétation des résultats **7**

9 Stockage, déploiement et tests **7**

10 Limites connues **7**

1 Principes de conception

1. **Le modèle ne calcule jamais.** Un petit modèle ne sait pas calibrer et invente des règles. Les méthodes de datation sont donc reprises de ChronoModel en C++ (`src/methods/`) et exécutées localement. Le modèle reçoit le *texte* des résultats exacts et les commente.

2. **Le modèle ne produit jamais le `.chr` complet.** Il écrit une spécification JSON courte (une dizaine de champs) ; `ChrBuilder` ajoute les quelque 200 champs obligatoires (identifiants, UUID, réglages MCMC, plage d'étude, positions…) et contrôle la cohérence.

3. **Connaissances par fiches.** Le savoir-faire est rangé dans des *compétences* (fiches Markdown) choisies à chaque question selon ses mots-clés, dans un budget de texte. On enrichit l'application en ajoutant des fiches, sans recompiler.

4. **Un seul jeu de calculs pour deux moteurs.** Modèle local (bloc `"‘calcul`) et modèle distant (appel d'outils natif) déclenchent les mêmes fonctions ; l'historique est identique, on peut changer de moteur en cours de conversation.

5. **Confidentialité.** Avec le modèle local, rien ne quitte la machine. Avec un modèle distant, seuls la conversation, les fiches choisies, les pièces jointes et les paramètres d'appel d'outils sont envoyés ; les courbes de calibration et les calculs restent locaux. La clé API est rangée dans le trousseau du système (QtKeychain).

2 Graphe fonctionnel

La figure 1 montre les modules et les flux. Les flèches pleines sont les flux de données d'une question ; les flèches en tirets sont les boucles (relance du modèle après un calcul, retour des résultats de ChronoModel).

Rôle des modules

Module	Rôle
`MainWindow`	Orchestration d'une question (algorithme 1), gestion des tours, liens `seldon://` (enregistrer un `.chr`, un fichier de densité, rappeler une figure).
`ChatView`, `MathText`	Rendu Markdown → HTML, formules LaTeX du modèle → HTML.
`FigurePanel`, `ChrScene`	Courbes (densités, HPD remplie, superposition ou une densité par ligne), export PNG/SVG, scènes des Events et des phases d'un `.chr`.
`Calculator`	Calculatrice locale (« 2+3 », `ADtoBP(-550)`) sans appel au modèle.
`Lexicon`	Ramène une question en espagnol, italien ou allemand aux mots-clés français.
`SkillManager`	Lecture des fiches (en-tête : `keywords`, `priority`, `always`) et sélection.
`LlmCore` / `LlmEngine`	Enveloppe C++ de llama.cpp (sans Qt) ; adaptateur Qt dans un `QThread`.
`RemoteEngine` / `RemoteApi`	API Chat Completions compatible OpenAI, flux SSE, appel d'outils.
`AutoTune`	« Optimiser pour ce modèle » : contexte, longueur de réponse, budget des fiches.
`MethodRunner`, `Tools`	Détecteurs de datations, normalisation et validation des requêtes, exécution, fiche de résultat, texte pour le modèle ; définition des 9 outils.
`MethodRegistry`	Une classe `Method` par technique (c14, f14c, gauss, unif, tl, am).
`EventMcmc`	Estimation bayésienne d'un Event (modèle EDM2), diagnostics $\hat{R}$ et ESS.
`CurveStore`, `RefCurve`	Courbes de référence : dossier utilisateur, `Calib` de ChronoModel, courbes intégrées.
`ChrBuilder`	Spécification JSON → `.chr` valide ; contraintes en toutes lettres ; vérification croisée avec le texte de l'utilisateur.
`ChronoResults`	Lecture des logs et résultats de ChronoModel joints, résumé dans un budget.
`ConversationStore`	Historique (écriture atomique `QSaveFile`).

3 Algorithme général

L'algorithme 1 décrit le traitement d'une question, de la saisie à l'enregistrement de la conversation. Les étapes 1 à 4 sont des *courts-circuits* : elles répondent sans le modèle de langage.

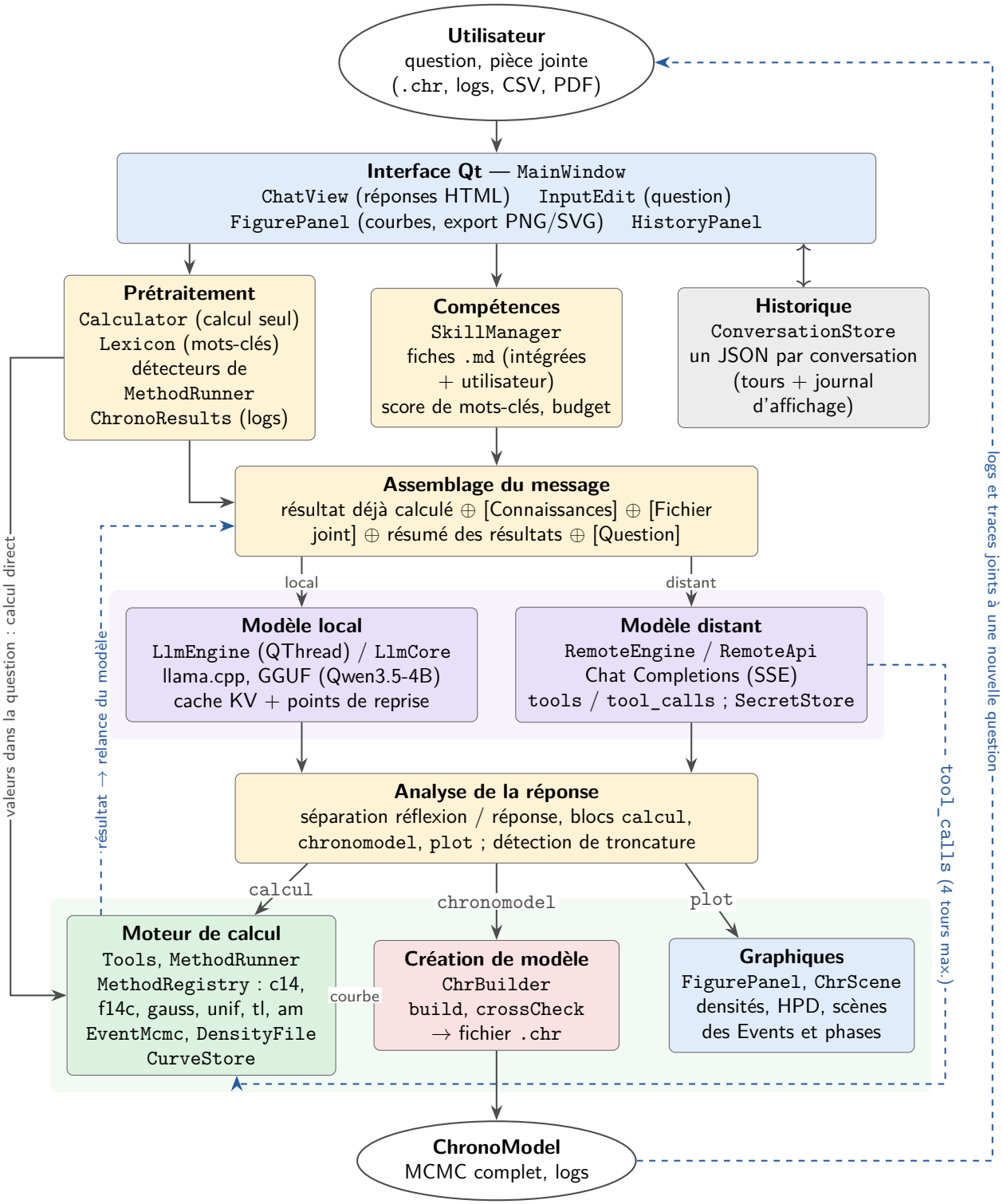


Figure 1: Graphe fonctionnel de SeldonH. La réponse est affichée en flux dans l'interface pendant sa génération. Orange : préparation et analyse du texte ; violet : moteurs de langage ; vert : calculs locaux déterministes ; rouge : création du modèle ChronoModel ; gris : stockage.

Algorithme 1 Traitement d'une question (`MainWindow::sendMessage` et suite)

Entrées : question q, pièce jointe éventuelle a, conversation C (tours précédents), réglages (format de date, budget B des fiches, moteur).
Sorties : réponse affichée, figures, éventuel fichier `.chr`, conversation enregistrée.

si $q =$ `/aide` **alors** afficher le mode d'emploi ; **fin**
si `Calculator.isCalculation`(q) **alors** afficher le résultat exact ; **fin** *rien n'est envoyé au modèle*
si q demande un dessin **et** un `.chr` est connu **alors** dessiner les scènes ;
  si la question ne demande que le dessin **alors fin**
si aucun modèle de langage n'est prêt **alors** message d'état ; **fin**
$q_k \leftarrow$ `Lexicon.translate`(q) *mots-clés en français*
$m \leftarrow$ `isModelRequest`(q_k) ; $M \leftarrow$ `datingMethodsIn`(q_k)
$P \leftarrow$ fiches imposées : 03 si m ; 13 si âge-profondeur ; fiche de chaque méthode de M ;
  19 (phases, Events) ou 11 (logs) si des résultats ChronoModel sont présents
$K \leftarrow$ `SkillManager.select`$(q_k,\ m\ ?\ 0 : B,\ \text{exclues},\ P)$ *voir §4*
$R \leftarrow$ `MethodRunner.analyzeQuestion`(q) *valeurs écrites dans la question*
si non R.trouvé **alors** $R \leftarrow$ recalcul de la datation précédente (autre niveau, autre période)
si R.trouvé **alors** afficher la fiche (HPD, médiane, mode) et la densité
$u \leftarrow R$.texte $\oplus\ K \oplus\ a \oplus$ résumé des résultats $\oplus\ q$ *message vu par le modèle*
ajouter le tour (q, u) à C ; enregistrer C
répéter *boucle agent*
  générer la réponse en flux (local : gabarit + cache KV ; distant : SSE avec déclaration des outils)
  tant que le moteur distant renvoie des `tool_calls` **et** tours ≤ 4 **faire**
    exécuter chaque outil localement (`Tools`) ; réinjecter le texte du résultat ; poursuivre
  afficher la réponse : réflexion séparée, blocs `calcul`/`chronomodel` masqués, `plot` $\rightarrow$ panneau
  si bloc `chronomodel` **alors**
    $S \leftarrow$ spécification ; `ChrBuilder.build`(S) ; `crossCheck`(S, q)
    afficher le résumé, les avertissements et le lien « Enregistrer le modèle (`.chr`) »
  si bloc `calcul` **et** le tour n'a pas déjà été calculé **alors**
    $R' \leftarrow$ `MethodRunner.run`(blocs) ; afficher fiche et figures
    ajouter un tour synthétique $u \leftarrow R'$.texte ; **continuer** *le modèle commente*
  sinon sortir de la boucle
si la réponse est coupée (limite de longueur, contexte plein, flux interrompu) **alors**
  proposer « Continuer la réponse » (la suite s'ajoute au même tour)
enregistrer C (tours + journal d'affichage)

Remarques.

- Un calcul est fait *au plus une fois* par question : un tour marqué « déjà calculé » (`followUp`) ne relance pas de calcul, ce qui évite les boucles avec le modèle local.
- Le modèle local n'a qu'un tour de suite après un bloc `calcul` ; le modèle distant peut enchaîner jusqu'à 4 appels d'outils, puis la requête est envoyée avec `tool_choice: none`.
- Si le serveur distant refuse le champ `tools`, la requête est rejouée sans outils et le bloc `calcul` prend le relais. Un appel d'outil est réécrit en bloc `calcul` dans l'historique.
- Les fiches des tours passés ne sont pas renvoyées : seul le tour courant les reçoit, ce qui garde le prompt court.

4 Sélection des compétences

Chaque fiche `.md` porte un en-tête (`name`, `description`, `keywords`, `priority`, `always`). La fiche d'identité (`always: true`) va dans le message système ; les autres sont placées devant la question. Le score d'une fiche pour une question q (sans accents, en minuscules) est

$$s = 10\Big(\sum_{k \in \text{mots-clés}} w(k) + \sum_{w \in \text{description}} [\, w \in q \,]\Big) + \text{priorité},$$

$$w(k) = \begin{cases} 4 & k \text{ expression trouvée dans } q \\ 3 & k \text{ mot de } q \\ 2 & |k| > 4 \text{ et } k \text{ sous-chaîne de } q \text{ (pluriel, préfixe)}, \\ 0 & \text{sinon.} \end{cases}$$

Les fiches imposées passent d'abord, puis les fiches par score décroissant tant que le budget (6000 caractères par défaut) n'est pas dépassé. Pour une création de modèle, le budget est mis à zéro : seules la fiche commune (03) et les fiches des méthodes citées sont fournies (exemple réel : 6 100 caractères au lieu de 18 500). Une fiche utilisateur de même nom remplace la fiche intégrée.

5 Moteurs de langage

5.1 Modèle local (llama.cpp)

`LlmCore` formate la conversation avec le gabarit du modèle, n'évalue que le texte nouveau (cache KV), génère en flux et oublie les plus anciens échanges si le contexte est plein. Modèle recommandé : Qwen3.5-4B en Q4_K_M (≈ 2,7 Go, Apache 2.0), réflexion désactivée par défaut (préfixe `<think></think>` vide).

Modèles hybrides. Qwen3.5 mêle couches récurrentes (Gated DeltaNet) et couches d'attention ; l'état récurrent ne peut pas être raccourci. `LlmCore` sauve donc au plus trois *points de reprise* de l'état récurrent (≈ 50 Mo chacun) : fin du message système, début et fin du dernier message utilisateur. Au tour suivant, on restaure le point le plus avancé contenu dans le préfixe commun, puis on raccourcit les couches d'attention.

5.2 Modèle distant

`RemoteEngine` parle à toute API Chat Completions compatible OpenAI (Base URL + clé). Réponse en flux SSE ; troncature détectée par `finish_reason: "length"` ou par l'absence de `[DONE]`. Les 9 outils (`calibrer_radiocarbone`, `calibrer_f14c`, `dater_luminescence`, `densite_gauss`, `densite_uniforme`, `dater_archeomagnetisme`, `calculer_evenement`, `creer_densite`, `courbes_disponibles`) sont déclarés avec leur schéma ; leurs arguments sont validés et normalisés (virgule décimale, nombres écrits en texte) et une valeur manquante renvoie une consigne au modèle sans lancer de calcul.

6 Calculs de datation

6.1 Dates isolées

Chaque méthode (`src/methods/<id>/`) reprend la vraisemblance du plugin ChronoModel correspondant ; la densité est calculée sur la grille de la courbe de référence avec un a priori uniforme, normalisée par la méthode des trapèzes, tronquée à 10^{-5} ; les régions HPD (95 % et 68 % par défaut) sont obtenues par dichotomie sur le seuil de densité. La fiche affiche HPD, médiane, mode, et la densité est tracée dans le panneau de droite. Les décimales affichées suivent l'écart-type (deux chiffres significatifs) ou un réglage fixe.

6.2 Calcul d'un Event (MCMC)

`EventMcmc` porte le moteur de ChronoModel 4.0 pour un Event sans contrainte, modèle EDM2 :

$$y_i = t_i + \delta_i, \qquad t_i \sim \mathcal{N}(\theta, \sigma_i^2), \qquad \theta \sim \mathcal{U}[t_{\min}, t_{\max}], \qquad \sigma_i^2 \mid S_0^2 \sim \text{shrinkage}(S_0^2), \qquad S_0^2 \sim \text{Inv-Gamma}(1, \beta).$$

Algorithme 2 Échantillonnage d'un Event (`EventMcmc`, d'après `MCMCLoopChrono`)

pour chaque chaîne $c = 1..N_c$ (fil d'exécution séparé, graine propre) **faire**
  initialiser t_i, σ_i, θ (`initialize_time`)
  chauffe : `num_burn` itérations
  adaptation : par lots, jusqu'à ce que tous les taux d'acceptation soient dans leur fenêtre
    (42–46 % ; 30–40 % pour t_i, σ_i d'un Event à plusieurs dates), Robbins–Monro
  acquisition : `num_iter` itérations, éclaircissement `thinning`
    pour chaque date i : bloc $(t_i, \sigma_i, \delta_i)$ par Metropolis–Hastings (calibration + shrinkage)
    décalage rigide $\theta + t_i$; Gibbs de θ : $\mathcal{N}(\mu, 1/P)$ tronquée à la période d'étude
    marche adaptative sur $\log_{10} S_0^2$; recuit périodique ($T = 2^e$, e décroissant)
densités a posteriori de θ, t_i : noyau gaussien, largeur de bande de Sheather–Jones
diagnostics : split-$\hat{R}$ rang-normalisé (bulk et tail), ESS bulk et tail
verdict : « non convergé » si $\hat{R} \geq 1{,}05$ ou ESS $< 100N_c$; « satisfaisant » si $\hat{R} < 1{,}01$ et ESS $\geq 500N_c$

6.3 Autres calculs

- `DensityFile` : fichier de densité pour le plugin Density de ChronoModel (lois normale, log-normale, uniforme, triangulaire, trapézoïdale, exponentielle, gamma, bêta ; placement $t = \text{offset} \pm x$).
- `Combination`, `Traces`, `InverseCalibration` : combinaison de traces MCMC jointes et calibration inverse.
- `CurveStore` : courbes cherchées dans le dossier utilisateur, puis dans `Calib` de ChronoModel, puis dans les courbes intégrées (`.14c`, `.ref`).

7 Création d'un modèle ChronoModel

Le modèle de langage écrit un bloc `"‘chronomodel` : Events, dates (`type` : `gauss`, `unif`, `tl`, `c14`, `f14c`, `am`, `density`, avec `wiggle` facultatif), bornes, phases et contraintes. `ChrBuilder` :

1. refuse les erreurs bloquantes (borne écrite comme une date, ancien nom de champ, TL sans `ref_year`) ;
2. traduit les contraintes écrites en toutes lettres (« US 12 sous US 10 », « coupe », « scellé par »…) en couples (`bwd_id` = plus ancien, `fwd_id` = plus récent), détecte contradictions et cycles ;
3. calcule la plage d'étude (HPD 95 % calibrée pour 14C et F14C, plus une marge) ;

4. complète les valeurs relevées dans des projets ChronoModel 4.0 réels (échantillonneurs, réglages MCMC, couleurs, positions) ;

5. compare la spécification au texte de l'utilisateur (`crossCheck`) : valeurs absentes du texte, références inventées, mesures du texte oubliées.

Le résultat s'affiche comme un résumé en clair et un lien d'enregistrement ; le JSON n'est montré que sur demande.

8 Interprétation des résultats

Les logs de ChronoModel joints (`Log_Model_Description`, `Log_MCMC_Initialization`, `Log_MCMC_Adaptation`, `Log_Posterior_Distrib_Stats`) sont lus par `ChronoResults`, résumés dans un budget adapté au contexte du modèle, et accompagnés de la fiche de diagnostic. La démarche imposée au modèle : verdict global, liste des variables en échec (Event → donnée → grandeur), localisation de la cause, distinction entre problème de *modèle* (contrainte inversée, dates incompatibles) et de *réglage MCMC*, actions chiffrées, contrôle après relance. Les tableaux de phases et d'Events sont produits par l'application avec les valeurs exactes, le modèle les commente.

9 Stockage, déploiement et tests

- **Historique** : un fichier JSON par conversation (`turns` pour reprendre le dialogue avec n'importe quel moteur, `display` pour redessiner l'écran), enregistré après la question, à chaque fin de réponse, en cas d'erreur et à la fermeture.

- **Déploiement** : un installeur Qt Installer Framework par système (Windows, Debian/Ubuntu, macOS), avec ou sans modèle (`_noModel`). Windows et Linux : llama.cpp en bibliothèques partagées, 14 variantes CPU + Vulkan chargées dynamiquement ; macOS : statique universel avec Metal.

- **Tests** : `tests/tst_core` (109 tests au 26/10/2026 : compétences, `ChrBuilder` contre des projets réels, méthodes, Sheather–Jones, formats de date) et `tests/llm_cli` (gabarit, cache KV, points de reprise, troncature).

10 Limites connues

- Les calculs intégrés traitent des dates isolées et un Event sans contrainte ; ils ne remplacent pas ChronoModel pour un modèle complet (phases, contraintes, wiggle matching).

- Codes `process_type` 4, 5, 6 (ID, IF, IDF) de l'archéomagnétisme supposés ; import dans ChronoModel de `.chr` 14C, F14C et density générés à confirmer.

- Appel d'outils testé contre un serveur factice uniquement ; API Anthropic native non prise en charge.

- Aucun banc d'essai des modèles de langage sur des tâches de datation réelles.